

# Intel HD Audio

## Controller & Codec Register Reference

Distilled from Intel "High Definition Audio Specification" Rev. 1.0a (June 17, 2010) and Linux kernel `include/sound/hda_verbs.h`. Working notes for the `intel-hda` / `intel-hda-uefi` crates in this repo — not a substitute for the full spec (see `docs/reference/intel-hda-spec-1.0a.pdf`).

### 1 Controller register set (MMIO, BAR0)

All registers live at the memory-mapped BAR0 given by the HDA PCI function's BAR0 (32- or 64-bit, 16 KB region, non-prefetchable). Enable Memory Space and Bus Master in the PCI Command register before touching any of these.

| Offset | Size | Name       | Purpose                                                                     |
|--------|------|------------|-----------------------------------------------------------------------------|
| 0x00   | 2    | GCAP       | Global Capabilities: OSS(15:12) ISS(11:8) BSS(7:3) NSDO(2:1) 64OK(0)        |
| 0x02   | 1    | VMIN       | Minor version                                                               |
| 0x03   | 1    | VMAJ       | Major version (0x01 for 1.0/1.0a)                                           |
| 0x04   | 2    | OUTPAY     | Output payload capability                                                   |
| 0x06   | 2    | INPAY      | Input payload capability                                                    |
| 0x08   | 4    | GCTL       | bit8 UNSOL, bit1 FCNTRL, bit0 CRST (0=reset, 1=run)                         |
| 0x0C   | 2    | WAKEEN     | Per-SDIN wake/interrupt enable (bits 14:0)                                  |
| 0x0E   | 2    | STATESTS   | Per-SDIN codec-present / state-change flags (bits 14:0), RW1C               |
| 0x10   | 2    | GSTS       | bit1 FSTS (flush status), RW1C                                              |
| 0x18   | 2    | OUTSTRMPAY | Output stream payload capability                                            |
| 0x1A   | 2    | INSTRMPAY  | Input stream payload capability                                             |
| 0x20   | 4    | INTCTL     | bit31 GIE, bit30 CIE, bits 29:0 per-stream SIE                              |
| 0x24   | 4    | INTSTS     | bit31 GIS, bit30 CIS, bits 29:0 per-stream SIS (RO)                         |
| 0x30   | 4    | WALCLK     | 24 MHz wall-clock counter, free-running while BCLK is up                    |
| 0x38   | 4    | SSYNC      | Per-stream bit; set to hold a stream's DMA off the link for sync start/stop |
| 0x40   | 4    | CORBLCASE  | CORB physical base, low 32 bits (128-byte aligned)                          |
| 0x44   | 4    | CORBUBASE  | CORB physical base, high 32 bits                                            |
| 0x48   | 2    | CORBWP     | CORB write pointer (bits 7:0), Dword units                                  |
| 0x4A   | 2    | CORBRP     | CORB read pointer (bits 7:0, RO) / bit15 CORBRPRST                          |
| 0x4C   | 1    | CORBCTL    | bit1 CORBRUN, bit0 CMEIE                                                    |
| 0x4D   | 1    | CORBSTS    | bit0 CMEI, RW1C                                                             |
| 0x4E   | 1    | CORBSIZE   | bits 1:0 size select (10 = 256 entries), bits 7:4 capability                |
| 0x50   | 4    | RIRBLBASE  | RIRB physical base, low 32 bits                                             |
| 0x54   | 4    | RIRBUBASE  | RIRB physical base, high 32 bits                                            |
| 0x58   | 2    | RIRBWP     | RIRB write pointer (bits 7:0, RO) / bit15 RIRBWPRST (W)                     |
| 0x5A   | 2    | RINTCNT    | Responses-per-interrupt count                                               |
| 0x5C   | 1    | RIRBCTL    | bit2 RIRBOIC, bit1 RIRBDMAEN, bit0 RINTCTL                                  |
| 0x5D   | 1    | RIRBSTS    | bit2 RIRBOIS, bit0 RINTFL, RW1C                                             |
| 0x5E   | 1    | RIRBSIZE   | bits 1:0 size select (10 = 256 entries)                                     |
| 0x60   | 4    | ICOI       | Immediate Command Output (optional PIO path, unused if CORB/RIRB used)      |
| 0x64   | 4    | ICII       | Immediate Response Input                                                    |
| 0x68   | 2    | ICIS       | Immediate Command Status                                                    |

| Offset        | Size | Name      | Purpose                                 |
|---------------|------|-----------|-----------------------------------------|
| 0x70          | 4    | DPIBLBASE | DMA Position-in-Buffer array, low base  |
| 0x74          | 4    | DPIBUBASE | DMA Position-in-Buffer array, high base |
| 0x80 + n·0x20 | —    | SDnCTL... | Stream descriptor n block, see below    |

Stream descriptors start at 0x80: input streams (ISS of them) first, then output streams (OSS), then bidirectional (BSS). With ISS input streams, output stream 0's block starts at  $0x80 + ISS \cdot 0x20$ .

### 1.1 Stream descriptor block (32 bytes per stream)

| Offset | Size           | Name             | Purpose                                     |
|--------|----------------|------------------|---------------------------------------------|
| +0x00  | 4 <sup>1</sup> | SDnCTL / STS     | see bit layout below                        |
| +0x04  | 4              | SDnLPIB          | Link Position in Buffer (RO)                |
| +0x08  | 4              | SDnCBL           | Cyclic Buffer Length in bytes               |
| +0x0C  | 2              | SDnLVI           | Last Valid Index into the BDL (0-based)     |
| +0x0E  | 2              | SDnFIFOW         | FIFO watermark (input streams)              |
| +0x10  | 2              | SDnFIFOS / FIFOD | FIFO size                                   |
| +0x12  | 2              | SDnFMT           | Stream format, see below                    |
| +0x18  | 4              | SDnBDPL          | BDL pointer, low 32 bits (128-byte aligned) |
| +0x1C  | 4              | SDnBDPU          | BDL pointer, high 32 bits                   |

**SDnCTL/STS bit layout** (32-bit word at +0x00):

|                  |                                                                                                |
|------------------|------------------------------------------------------------------------------------------------|
| bits 31:24 (STS) | bit29(5) FIFORDY · bit28(4) DESE (RW1C) · bit27(3) FIFOE (RW1C) · bit26(2) BCIS (RW1C)         |
| bits 23:20       | STRM — stream tag (1-15; 0 = unused/reserved)                                                  |
| bit 19           | DIR — bidirectional streams only: 0 = input, 1 = output                                        |
| bit 18           | TP — traffic priority                                                                          |
| bits 17:16       | STRIPE — 00 = 1 SDO, 01 = 2 SDOs, 10 = 4 SDOs                                                  |
| bit 4            | DEIE — descriptor error interrupt enable                                                       |
| bit 3            | FEIE — FIFO error interrupt enable                                                             |
| bit 2            | IOCE — interrupt-on-completion enable                                                          |
| bit 1            | RUN — start/stop the DMA engine                                                                |
| bit 0            | SRST — stream reset (write 1, poll until read-back 1, then write 0 and poll until read-back 0) |

**SDnFMT bit layout** (16 bits, PCM Format Structure, spec §3.7.1):

|            |                                                               |
|------------|---------------------------------------------------------------|
| bit 15     | TYPE — 0 = PCM, 1 = non-PCM                                   |
| bit 14     | BASE — 0 = 48 kHz, 1 = 44.1 kHz                               |
| bits 13:11 | MULT — 000=×1 001=×2 010=×3 011=×4                            |
| bits 10:8  | DIV — 000=÷1 001=÷2 010=÷3 011=÷4 100=÷5 101=÷6 110=÷7 111=÷8 |
| bits 6:4   | BITS — 000=8b 001=16b 010=20b 011=24b 100=32b                 |
| bits 3:0   | CHAN – 1 (0000 = 1 channel, 0001 = 2 channels, ...)           |

Example: 48 kHz / 16-bit / 2ch = BASE=0 MULT=000 DIV=000 BITS=001 CHAN=0001 = 0x0011.

### 1.2 Buffer Descriptor List (BDL) entry (16 bytes each, up to 256 entries)

| Offset | Size | Field                                                                                                                 |
|--------|------|-----------------------------------------------------------------------------------------------------------------------|
| +0x00  | 8    | Address — physical address of this buffer fragment (must not cross a 4 GB boundary if only 32-bit addressing is used) |

<sup>1</sup>Spec lists CTL as 3 bytes + STS as 1 byte; both are commonly accessed together as one 32-bit register with STS in bits 31:24.

| Offset | Size | Field                                                                               |
|--------|------|-------------------------------------------------------------------------------------|
| +0x08  | 4    | Length — fragment length in bytes                                                   |
| +0x0C  | 4    | bit 0 IOC — Interrupt On Completion when this fragment finishes; bits 31:1 reserved |

There must be at least 2 valid entries ( $LVI \geq 1$ ). SDnCBL is the sum of all fragment lengths in the active list and must be an integer number of samples.

## 2 Controller bring-up sequence

1. Enable Memory Space + Bus Master in the PCI Command register; program the PCI BAR/interrupt line as usual.
2. Write GCTL.CRST = 0, confirm read-back 0 (controller enters reset), wait, then write GCTL.CRST = 1 and poll until it reads back 1 (link out of reset).
3. Wait  $\geq 521 \mu s$  (25 frames) after CRST reads 1, so that attached codecs finish requesting their addresses.
4. Read STATESTS: each set bit  $n$  means a codec responded on SDIN  $n$ .
5. Allocate CORB (up to 1 KB,  $256 \times 4$ -byte entries) and RIRB (up to 2 KB,  $256 \times 8$ -byte entries) as physically-contiguous, 128-byte-aligned DMA memory. Program CORBLBASE/UBASE and RIRBLBASE/UBASE (only while CORBRUN/RIRBDMAEN are 0), set CORBSIZE/RIRBSIZE to the 256-entry option, reset both pointers (CORBRP bit15, RIRBWP bit15 — write 1, the CORB one must read back 1 before clearing), then set CORBCTL.CORBRUN = 1 and RIRBCTL.RIRBDMAEN = 1.
6. For each codec address found in STATESTS, send Get Parameter (Vendor ID) to node 0 (root) to confirm it's alive, then walk the node tree (§ below).

## 3 Codec verb encoding (CORB entry, 32 bits)

bits 31:28 Codec Address (CA<sub>d</sub>)

bits 27:20 Node ID (NID)

bits 19:0 Verb payload

The 20-bit payload splits one of two ways depending on the verb:

- **12-bit verb + 8-bit payload** (most verbs — Get/Set Parameter, Connection Select, Power State, Pin Widget Control, Channel/Streamid, Pin Sense, EAPD, Unsolicited Response enable, ...):  $\text{payload}_{20} = (\text{verb}_{12} \ll 8) \mid \text{payload}_8$ .
- **4-bit verb + 16-bit payload** (Converter Format, Amplifier Gain/Mute, Processing Coefficient, Coefficient Index only):  $\text{payload}_{20} = (\text{verb}_4 \ll 16) \mid \text{payload}_{16}$ .

RIRB entry (64 bits, written by hardware): bytes 0-3 = the raw 32-bit codec response; bytes 4-7 = Resp\_Ex (bits 3:0 = codec address the response came from, bit 4 = 1 if unsolicited).

## 4 Node addressing & discovery

Every codec has a Root Node (NID 0) → one or more Function Group nodes (Audio Function Group = type 0x01) → widget nodes (converters, pins, mixers, selectors, ...). Get Parameter works on any node:

| Param ID    | Name                      | Response layout                                                            |
|-------------|---------------------------|----------------------------------------------------------------------------|
| 0x00        | Vendor ID                 | 31:16 Vendor ID · 15:0 Device ID                                           |
| 0x04        | Subordinate Node Count    | 23:16 Starting Node Number · 7:0 Total Number of Nodes                     |
| 0x05        | Function Group Type       | 8 UnsolCapable · 7:0 NodeType (0x01 = Audio Function Group)                |
| 0x08        | Audio Function Group Caps | 16 BeepGen · 11:8 InputDelay · 3:0 OutputDelay                             |
| 0x09        | Audio Widget Capabilities | see below                                                                  |
| 0x0A        | Supported PCM Size/Rates  | 20:16 bit depths B8/B16/B20/B24/B32 · 11:0 rates R1..R12                   |
| 0x0B        | Supported Stream Formats  | bit2 AC3 · bit1 Float32 · bit0 PCM                                         |
| 0x0C        | Pin Capabilities          | see below                                                                  |
| 0x0D / 0x12 | Amp Caps (in / out)       | 31 MuteCapable · 22:16 StepSize · 14:8 NumSteps · 6:0 Offset               |
| 0x0E        | Connection List Length    | 7 LongForm · 6:0 Length                                                    |
| 0x0F        | Supported Power States    | 31 EPSS · 30 CLKSTOP · 4 D3coldSup · 3 D3Sup · 2 D2Sup · 1 D1Sup · 0 D0Sup |

**Audio Widget Capabilities (param 0x09):** bits 23:20 = Type (0=Audio Output/DAC, 1=Audio Input/ADC, 2=Mixer, 3=Selector, 4=Pin Complex, 5=Power, 6=Volume Knob, 7=Beep Generator, 0xF=Vendor), bit 10 PowerCntrl, bit 9 Digital, bit 8 ConnList, bit 7 UnsolCapable, bit 4 FormatOverride, bit 3 AmpParamOverride, bit 2 OutAmpPresent, bit 1 InAmpPresent, bit 0 Stereo.

**Pin Capabilities (param 0x0C):** bit 27 HBR, bit 24 DP, bit 16 EAPD Capable, bits 15:8 VRefControl, bit 7 HDMI, bit 5 InputCapable, bit 4 OutputCapable, bit 3 HeadphoneDriveCapable, bit 2 PresenceDetectCapable, bit 0 ImpedanceSenseCapable.

## 5 Key controls used to route audio to an output pin

All verb IDs below already include the family shift described above — use them directly as `payload20 = (verb << shift) | payload`.

| Control                     | Get   | Set             | Payload width | Payload bits                                                      |
|-----------------------------|-------|-----------------|---------------|-------------------------------------------------------------------|
| Get Parameter               | 0xF00 | —               | 8             | param ID                                                          |
| Connection Select           | 0xF01 | 0x701           | 8             | connection list index                                             |
| Get Connection List Entry   | 0xF02 | —               | 8             | offset n into list (4 short-form or 2 long-form entries per call) |
| Converter Format            | 0xA   | 0x2             | 16            | PCM Format Structure (§3.7.1)                                     |
| Amplifier Gain/Mute         | 0xB   | 0x3             | 16            | see Note below                                                    |
| Power State                 | 0xF05 | 0x705           | 8             | bits 3:0 PS-Set (0=D0 1=D1 2=D2 3=D3)                             |
| Converter Stream/Channel    | 0xF06 | 0x706           | 8             | 7:4 stream tag · 3:0 channel                                      |
| Pin Widget Control          | 0xF07 | 0x707           | 8             | bit7 H-Phn En · bit6 OutEnable · bit5 InEnable · 1:0 VRefEn       |
| Unsolicited Response Enable | 0xF08 | 0x708           | 8             | bit7 Enable · 5:0 Tag                                             |
| Pin Sense                   | 0xF09 | 0x709 (execute) | 8             | get: bit31 PresenceDetect (+bit30 ELDV digital)                   |
| EAPD/BTL Enable             | 0xF0C | 0x70C           | 8             | bit2 L-R Swap · bit1 EAPD · bit0 BTL                              |

**Amplifier Gain/Mute Set payload (16 bits):** bit15 SetOutputAmp, bit14 SetInputAmp, bit13 SetLeft, bit12 SetRight, bits11:8 Index (input amp only), bit7 Mute, bits6:0 Gain. Setting both channel bits (13,12) programs both channels in one verb; setting neither is a no-op.

## 6 Minimal path to sound

1. Enumerate: Root → Audio Function Group → widgets. For each widget, read Widget Capabilities (0x09); collect DAC (type=0) and Pin Complex (type=4) nodes.
2. For each Pin Complex with OutputCapable set, read its Connection List (if ConnList capability bit set) and find a DAC on it — or just accept a fixed-function pin/DAC pair reported directly.
3. Set the DAC's Converter Format to match the stream you'll program into the Stream Descriptor (they must match exactly).
4. Set the DAC's Converter Stream/Channel to (stream\_tag, 0).
5. Set the DAC's Amplifier Gain/Mute (output amp, both channels, mute=0, reasonable gain) if OutAmpPresent.
6. Set the Pin Complex's Pin Widget Control: OutEnable=1 (and H-Phn Enable=1 if it's a headphone pin with that capability).
7. Set the Pin Complex's Amplifier Gain/Mute (unmute) if it has one, and EAPD=1 if EAPD Capable.
8. Program the controller's output Stream Descriptor: SDnFMT to the same format, SDnCTL.STRM = stream\_tag, build a BDL over your PCM buffer (looped by wrapping LVI), SDnCBL = buffer length, SDnLVI = last valid BDL index, SDnBDPL/U = BDL physical address, then set RUN = 1.

## 7 Implementation gotchas found while bringing this up

- **RINTCNT must be nonzero before any command will be processed, on at least one common emulation.** The spec's own text implies RINTCNT (the RIRB response-interrupt count) only controls when the interrupt flag gets set, not whether the CORB DMA engine runs at all. QEMU's ich9-intel-hda/intel-hda models it differently: their CORB-processing loop bails out whenever `responses_since_last_service ==`

RINTCNT, and since that counter starts at 0 and RINTCNT's own reset value is 0, the condition is true from the very first command – nothing is ever fetched until software programs RINTCNT to something nonzero. Fix: set RINTCNT to a nonzero value (1 works) during RIRB bring-up, enable RIRBCTL's interrupt-enable bit (RINTCTL) even if you never handle interrupts, and write a 1 to RIRBSTS's RINTFL bit after consuming every response – clearing a status bit you never handle is always safe on real hardware, and it's what resets that same counter back to 0 on the emulation, keeping command processing unblocked indefinitely rather than for only the first RINTCNT commands.

- **An amplifier gain/mute set of “gain 0” is not necessarily “0 dB”.** The Amplifier Capabilities parameter's offset field is what maps to 0 dB – not 0. A codec that doesn't implement Amplifier Capabilities at all (again, QEMU's built-in HDA codec: Get Parameter on 0x0D/0x12 just isn't in its parameter table, so it responds with all-zero) will report NumSteps = 0, Offset = 0, and driving that widget's gain to the “offset” value in that case sets an explicit gain of 0 – the quietest possible setting on hardware that **does** implement variable gain, and a no-op change in state on hardware that doesn't. Since a codec's own reset default is required to be unmuted at a sane gain already, the safe rule is: only issue an explicit Amplifier Gain/Mute **Set** when NumSteps > 0; otherwise leave the widget's default alone.

## 8 References

- docs/reference/intel-hda-spec-1.0a.pdf — Intel High Definition Audio Specification, Revision 1.0a (June 17, 2010). Authoritative source for everything above; see chapters 3 (Register Interface), 4 (Programming Model), and 7.3 (Codec Parameters and Controls).
- docs/reference/linux-hda\_verbs.h — Linux kernel's AC\_VERB\_\*, AC\_PAR\_\*, and bit-field constants, cross-checked against the spec while writing this document.